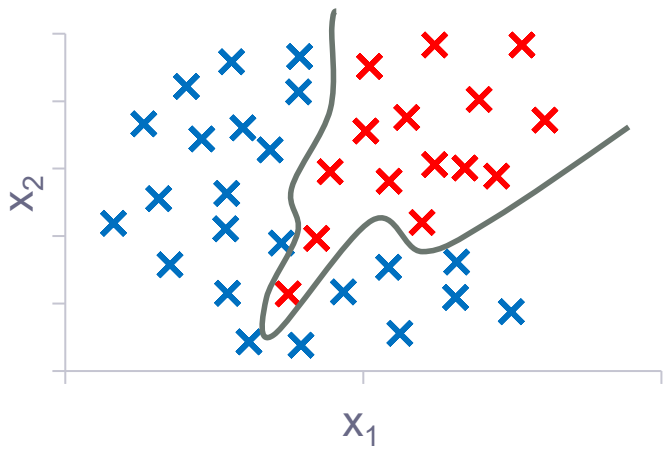
2. BASICS OF NEURAL NETWORKS

Prepared by Dr. Valy Dona

Non Linear Hypotheses

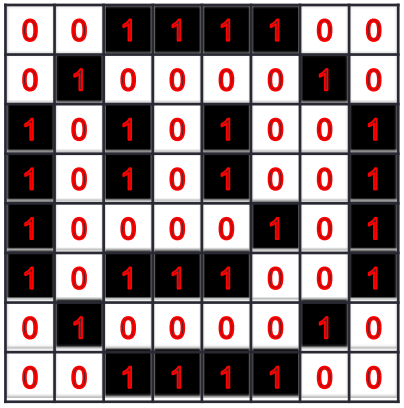
- $h(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1 x_2^2 + \cdots)$



Non Linear Hypotheses

- Image classification

8x8 pixel image

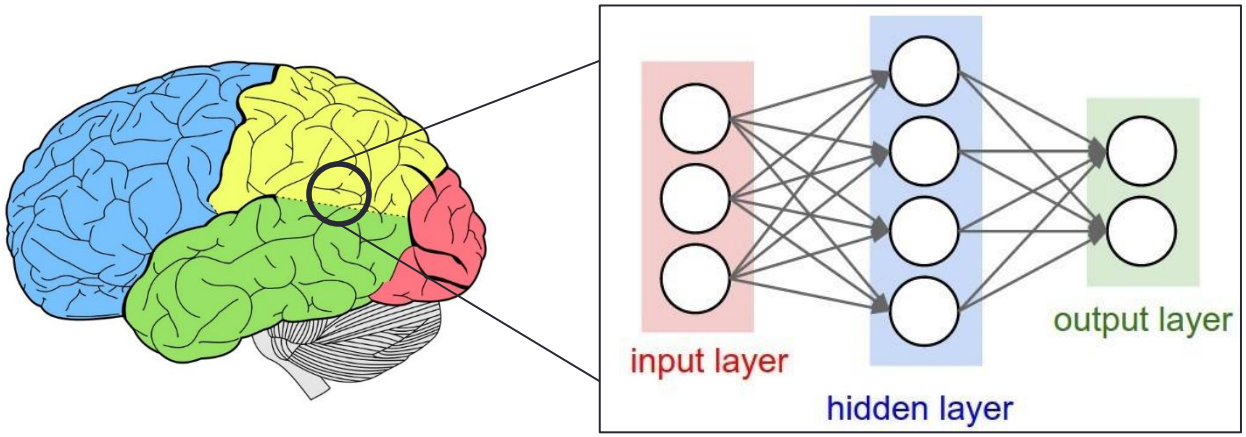


- 8x8 pixel image ⇒ 64 pixel values as features
- Number of features $n = 64$

$$x = \begin{bmatrix} pixel\ 1\ value \\ pixel\ 2\ value \\ \vdots \\ pixel\ n\ value \end{bmatrix}$$

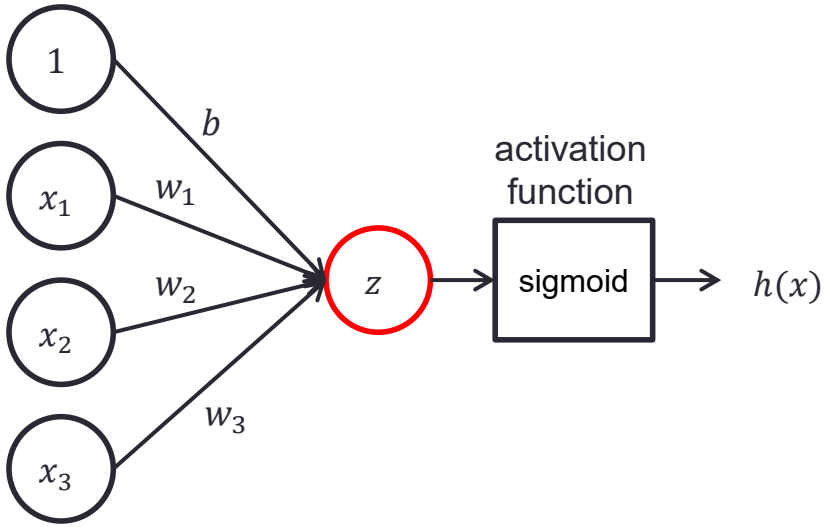
Artificial Neural Networks (ANN)

- **Idea:** to make the machine more robust, intelligent, and able to learn, let's model it after ***human brains***
- Biological Inspiration: the brain is composed of neurons
  - A neuron receives input from other neurons
  - Inputs are approximately summed
  - When the input exceeds a threshold the neuron sends an electrical spike that travels to the next neuron(s)



Neuron Model

- Logistic unit



$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$
$$h(x) = a = g(z)$$

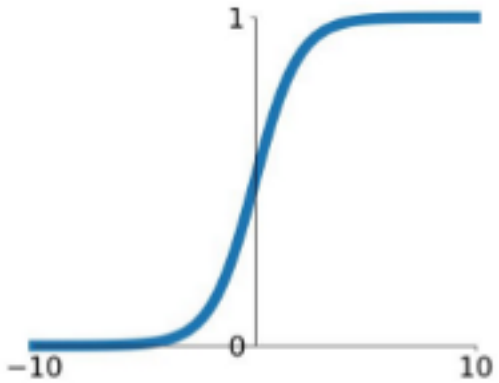
$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \; W = [w_1 \;\; w_2 \;\; w_3]$$

$$z = Wx + b$$

1x3 3x1

More on Activation Functions

- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$

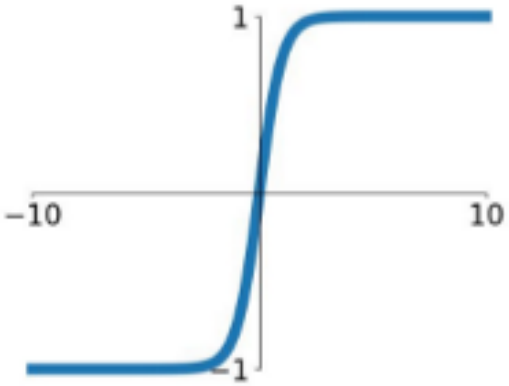


- Squashes numbers to range]0,1[

- ***Problems:***
  - *Saturated neurons "kills" the gradients*
  - *Sigmoid outputs are NOT zero-centered*
  - *Somehow computationally expensive*

More on Activation Functions

- Tanh: $\tanh(x)$



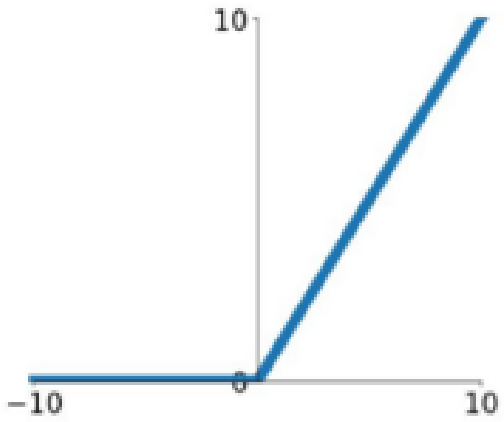
- Squashes numbers to range]-1,1[
- Zero-centered

- ***Problems:***
  - *Still "kills" the gradients*

More on Activation Functions

- Rectified Linear Unit (Relu):

$$\max(0, x)$$

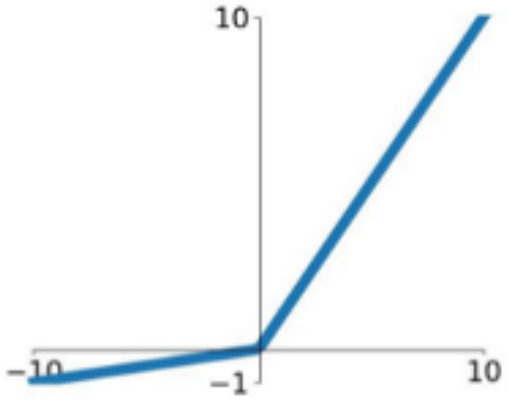


- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh

- ***Problems:***
  - *Not zero-centered output*
  - *What is the gradient when $x < 0$?*
    - *Neuron won't activate any more ⇒ weight not updated*

More on Activation Functions

- Leaky Relu: $\max(0.1x, x)$



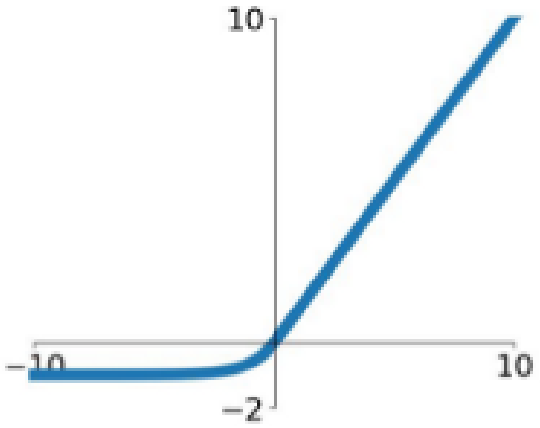
- Benefits of Relu
- Will not “die”

- Parametric Relu (PRelu): $\max(\alpha x, x)$

More on Activation Functions

- Exponential Linear Unit (ELU)

$$\begin{cases} x \text{ if } x \geq 0 \\ \alpha(e^x - 1) \text{ if } x < 0 \end{cases}$$

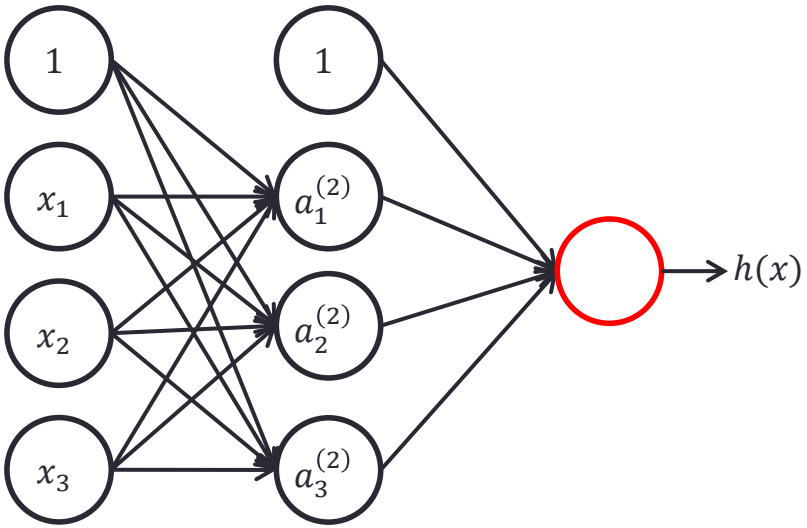


- Benefits of Relu/Leaky Relu
- Closer to zero mean output

- ***Problems:***
  - *Computation requires exp()*

Neural Network Model

- Neural network layers



Notation:

- $a_i^{(j)}$ = activation of unit i in layer j
- $\mathrm{W}^{(j)}$ = weight matrix mapping from layer j to layer $j+1$

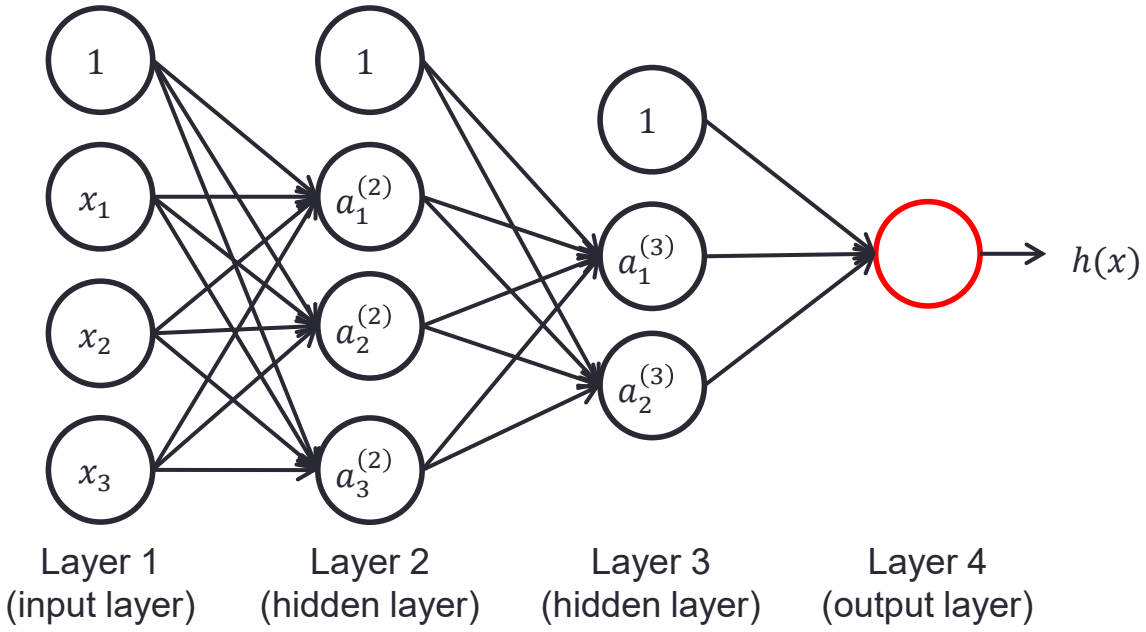
$$a^{(2)} = g(\mathrm{W}^{(1)}x + B^{(1)})$$

3x1 3x3 3x1 3x1

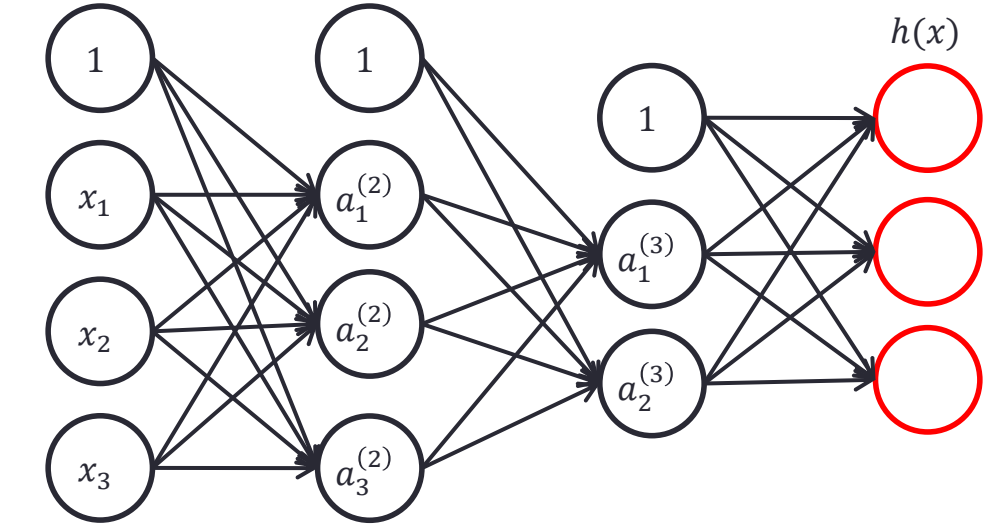
$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix},\ \mathrm{W}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \end{bmatrix},\ B^{(1)} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \end{bmatrix}$$

Neural Network Model

- Neural network layers



Neural Network for Multiclass Classification



y becomes ***one hot encoding***

It is either

$$\begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\end{bmatrix}$$

Neural Network for Multiclass Classification

$$y^{(i)} = \begin{bmatrix} y_1^{(i)} \\ y_2^{(i)} \\ \vdots \\ y_K^{(i)} \end{bmatrix}, h\left(x^{(i)}\right) = \begin{bmatrix} h\left(x^{(i)}\right)_1 \\ h\left(x^{(i)}\right)_2 \\ \vdots \\ h\left(x^{(i)}\right)_K \end{bmatrix}$$

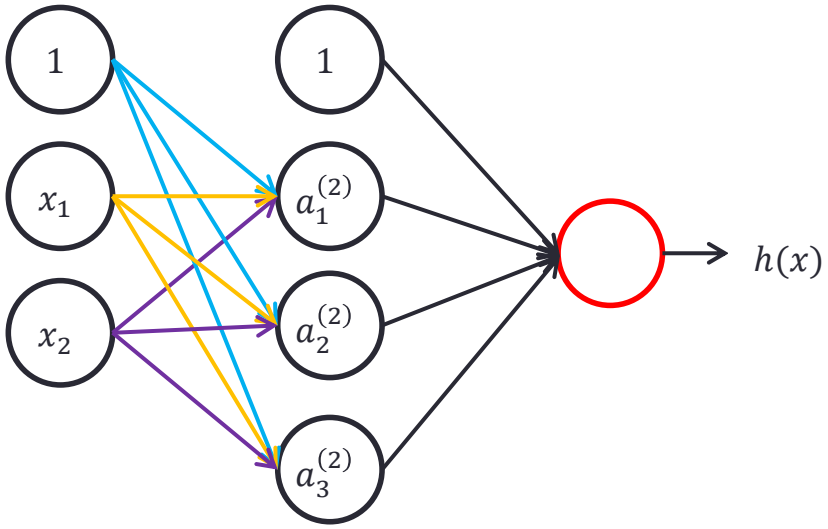
where $y_k^{(i)} = \{0,1\}$, K the number of classes

- Cost Function
  - $J(\Theta) = \frac{1}{m}\left[\sum_1^m \sum_1^K -y_k^{(i)} \log\left(h\left(x^{(i)}\right)_k\right) - \left(1 - y_k^{(i)}\right) \log\left(1 - h\left(x^{(i)}\right)_k\right)\right]$

- Note: the value of $h\left(x^{(i)}\right)_k$ should be $0 \leq h\left(x^{(i)}\right)_k \leq 1$

Parameter Initialization

- Zero initialization



After each update, parameters corresponding to inputs going into each of the hidden units are identical

$$\Rightarrow a_1^{(2)} = a_2^{(2)} = a_3^{(2)}$$

SOLUTION: Random Initialization (Symmetry Breaking)
$\Rightarrow$ Initialize each Θ_{ij} to a random value in $[-\epsilon, \epsilon]$ where ϵ is a small value

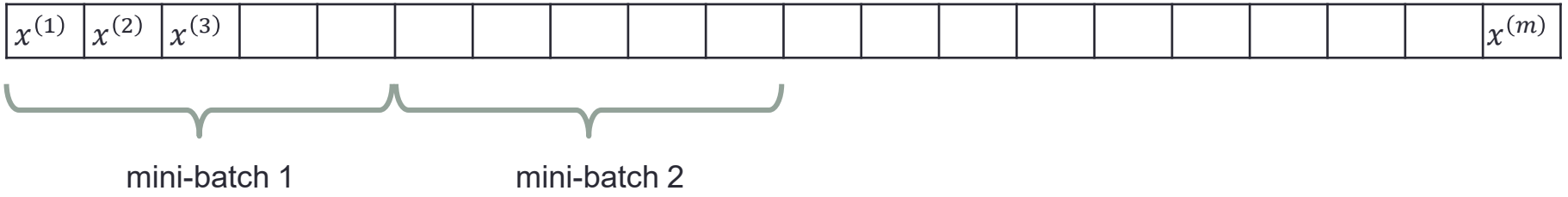
Summary: Building a Neural Network

- Pick a network architecture
- Randomly initialize parameters
- Implement forward propagation to get $h(x)$
- Implement cost function $J(\Theta)$
- Minimize $J(\Theta)$

- TRAINING LOOP: for each iteration
  - Perform forward propagation
  - Compute cost using $J(\Theta)$
  - Use gradient descent by implementing backpropagation to compute partial derivatives $\frac{\partial}{\partial \Theta_{ij}} J(\Theta)$
  - Update parameters

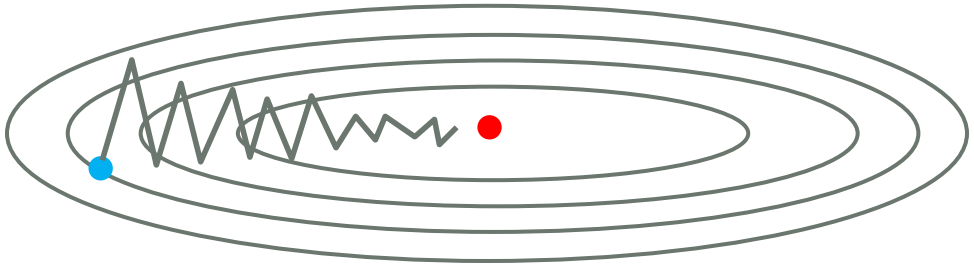
Mini-batch Gradient Descent

- 3 variants of gradient descent
  - **Batch GD:** update parameters after going through all training examples (1 epoch)
  - **Stochastic GD:** update parameters after each training example
  - **Mini-batch GD:** update parameters after a small batch of training examples (1 epoch might contain multiple mini-batches)



Optimizer

- Problem with GD



- GD:

  $\theta = \theta - \alpha grad_\theta$

- Momentum:

  $v_\theta = \beta v_\theta + (1 - \beta) grad_\theta$

  $\theta = \theta - \alpha v_\theta$

  - Now we have one more hyperparameter β ($\beta = 0.9$ in practice)

Optimizer

- RMSprop

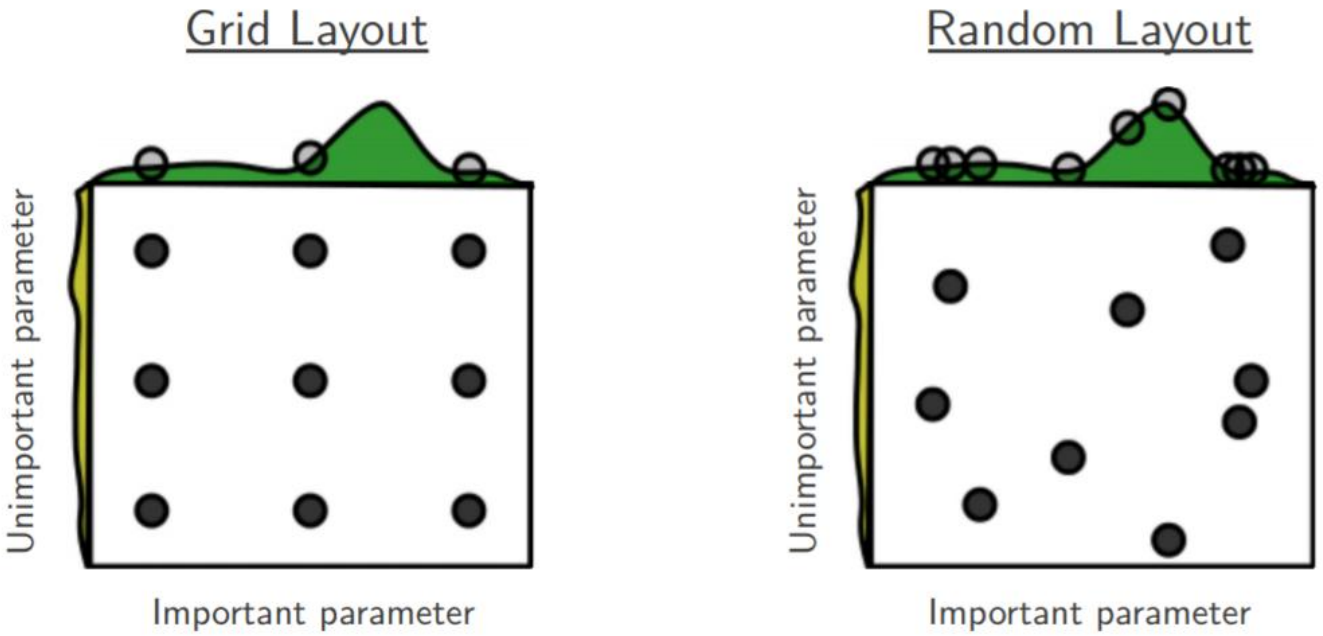
$$S_\theta = \beta S_\theta + (1-\beta) grad_\theta^2$$

$$\theta = \theta - \alpha \left(\frac{grad_\theta}{\sqrt{S_\theta}} \right)$$

- Adam: combine momentum and RMSprop

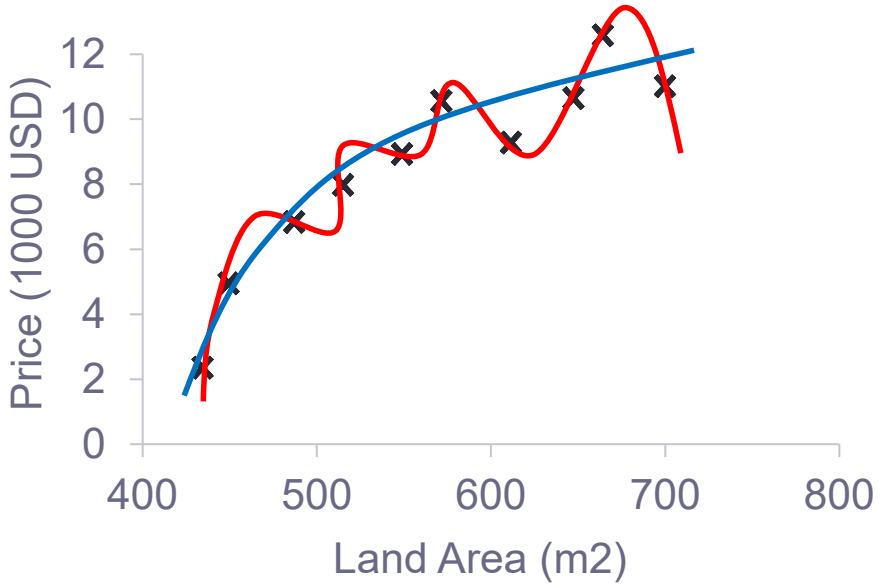
Hyperparameter Tuning

- Training/**Validation**/Test



Regularization

- Overfitting Problem



It fails to generalize to new examples not in the training set

Dataset with many input features?

Regularization

- Compare two hypothesis:

  $h_1(x) = \theta_0 + \theta_1 x + \theta_2 x^2$

  $h_2(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

  - We can penalize and make θ_3 and θ_4 really small

    $Loss \Rightarrow Loss + 1000\theta_3^2 + 1000\theta_4^2$

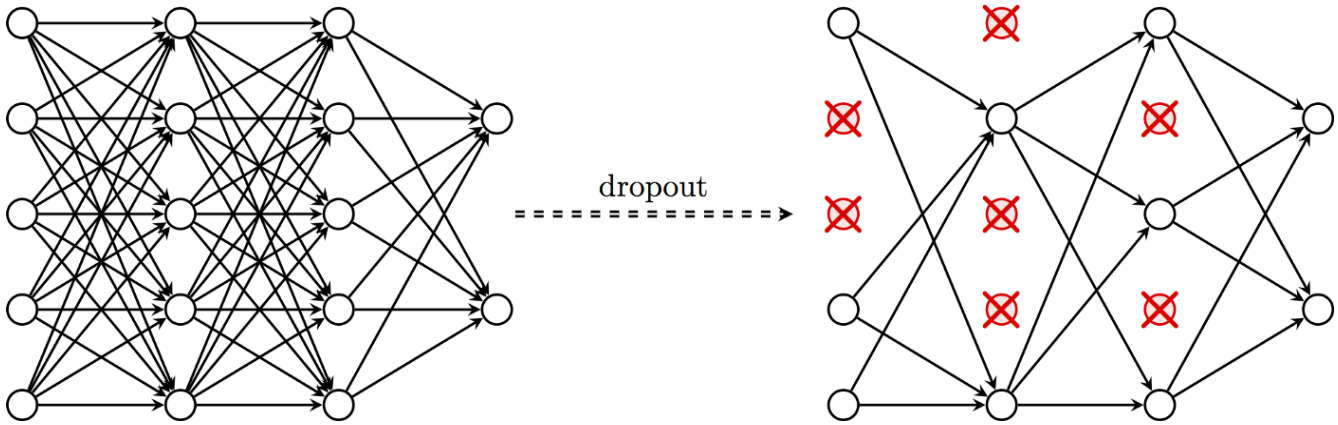
- Small values for parameters
  - Simpler hypothesis
  - Less prone to overfitting
  - The cost function now becomes:

    $Loss + \lambda \sum_{j=1}^{n} \theta_j^2$ (L2 Regularization)

    Or $Loss + \lambda \sum_{j=1}^{n} |\theta_j|$ (L1 Regularization)

Regularization

- Dropout: randomly set some neurons to zero in the forward pass



  - Forces the network to have a redundant representation (multiple important representations)
  - A large ensemble of models

- Ex: dropout of $p = 0.5$ (probability of a unit to be zeroed/dropped)

Batch Normalization

- Consider a batch of activation at some layer, to make each dimension unit gaussian, apply:
  - $\hat{z} = \frac{z-\mu}{s}$
- Problem: do we necessarily want a unit gaussian input to the activation function?
  - Do $\gamma\hat{z} + \beta$ where γ and β are trainable parameters
- Benefits of batch norm:
  - Improves gradient flow
  - Allows higher learning rate
  - Reduces strong dependence on initialization
  - Acts as a form of regularization

Exercise

Using the Khmer character dataset provided (kh_char.zip), construct a neural network and evaluate it by calculating the error rate on the test set. It is crucial to fine-tune your model and implement regularization techniques to maximize its performance.

References

- Ng, A. Coursera Machine Learning Course: coursera.org/learn/machine-learning
- Ng, A. (2018). Stanford CS229: Machine Learning.
- Karpathy, A. (2016). Stanford Winter Class CS231n: Convolutional Neural Networks for Visual Recognition.